

Introduction to Python Workshop

Save Humans in the AI Farm!

Mission Captains: Additi & Malindi



Mission Briefing:

Congratulations!

Your clearance has been approved.
Please write your names on the paper in front of you.

- You have been undercover and have been hired as a consultant for a farm harvesting human ideas run by AI.
- You need to learn enough Python to determine whether humans can make it out alive.

Mathematical Model of the Farm Dynamics

$$\dot{x} = rx\left(1 - \frac{x}{k}\right) - h\frac{x}{a + x}$$

x

human idea
population

r

growth rate

k

carrying capacity

h

harvest rate

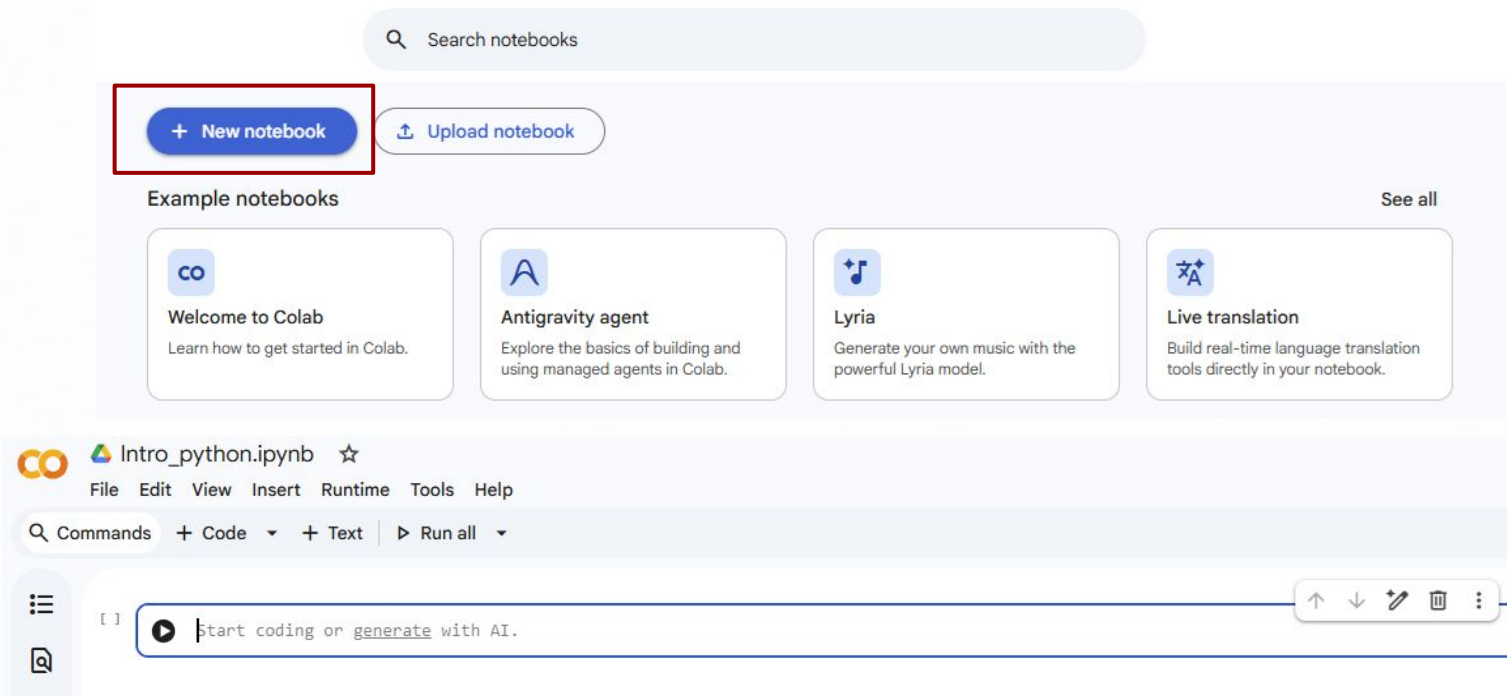
a

harvest saturation

Main question:
Will the humans survive?

Setup your workstation

Search 'Google colab' and click on 'https://colab.research.google.com/'



Recruit Specialists: Importing Packages

- A package is a library of codes that someone else wrote to provide tools/functions that Python doesn't already have built in.
- Use 'import' to get entire package or 'from ... import' to get specific tool.
- Common packages/aliases: numpy/np, matplotlib.pyplot/plt.

```
import {package name} as {alias}  
from {package name} import {function}
```

Hire your interns

1. Import numpy as np and matplotlib.pyplot as plt
2. Run the command: `print(np.__version__)`

```
# Import necessary packages
import numpy as np
import matplotlib.pyplot as plt
print(np.__version__)
```

Numpy: numerical python - good at numbers but overkill for small tasks.

Matplotlib: mathematical plotting library- makes evidence presentable.

Collect your ID cards: Data Types

int

3

count

float

2.79

decimal

bool

True

yes/no

str

"Apple"

Text

AI Personnel Database

```
age = 25 #int
height = 5.5 #float
human = True #bool
name = "Malindi" #string
type(age) #returns data type of variable
```

Collect your ID cards: Data Types

int

3

count

float

2.79

decimal

bool

True

yes/no

str

"Apple"

Text

AI Personnel Database

```
age = 25 #int
height = 5.5 #float
human = True #bool
name = "Malindi" #string
type(age) #returns data type of variable
```

Should return **int**

Create & Initialize Farm

1. Create and initialize r , K , h , a ($r = 0.5$, $K = 10$, $h = 1.5$, $a = 1$)
2. Create and initialize the initial population x_0 and a string `farm_name`
3. Create a bool variable that tests if the initial population is greater than 4
4. Print bool variable and check the type of 2 variables

Create & Initialize Farm: Solution

1. Create and initialize r, K, h, a (r = 0.5, K= 10, h =1.5, a=1)
2. Create and initialize the initial population x_0 and a string farm_name
3. Create a bool variable that tests if the initial population is greater than 4
4. Print bool variable and check the type of 2 variables

```
# Defining and initializing variables
```

```
r = 0.5
```

```
K = 10
```

```
h = 1.5
```

```
a = 1
```

```
x_0 = 6
```

```
farm_name = "AI-7"
```

```
viable = x_0 > 4
```

```
print(viable)
```

```
print(type(viable), type(farm_name))
```

```
True
```

```
<class 'bool'> <class 'str'>
```

Communication Protocol: Print Formatting

- Allows you to incorporate variables into the string you print
- Put variables inside { }
- Use: to apply specific rule like
 - `:.#f` rounds float to # decimal places
 - `:.#%` percentage with # decimal places
 - `#` symbol just means comments!

```
print(f"Hello my name is {name}, and I am  
{height:.1f} feet tall")
```

Communication Protocol: Print Formatting

- Allows you to incorporate variables into the string you print
- Put variables inside { }
- Use: to apply specific rule like
 - `:.#f` rounds float to # decimal places
 - `:.#%` percentage with # decimal places
 - `#` symbol just means comments!

```
print(f"Hello my name is {name}, and I am  
{height:.1f} feet tall")
```

```
Hello my name is Malindi, and I am  
5.5 feet tall
```

Quarterly report

3 min

1. Using your variables print: Farm AI-7: idea population 6.00 (60% of capacity)
2. Print all your system parameters in on line ($r = 0.5$, $K = 10$, $h = 1.5$, $a = 1$)

Quarterly report: Solution

1. Using your variables print: Farm AI-7: idea population 6.00 (60% of capacity)
2. Print all your system parameters in on line ($r = 0.5$, $K = 10$, $h = 1.5$, $a = 1$)

```
print(f"Farm {farm_name}: population {x_0:.2f} ({x_0/K:.0%} of capacity)")  
print(f"r = {r}, K = {K}, h = {h}, a = {a}")
```

```
Farm AI-7: population 6.00 (60% of capacity)  
r = 0.5, K = 10, h = 1.5, a = 1
```

Automating Skills: Defining Functions

- Reusable chunks of code
- Default arguments make input optional
- Inside functions passed in (immutable) objects only have local scope

```
def function_name(input_1, input_2=7):  
    #code here  
    return output
```


Dictate how the farm evolves

1. Write $\text{harvest}(x, h, a)$ and $\text{growth}(x, r, K)$
2. Write $\text{rhs}(t, x, r, K, h, a) = \text{growth} - \text{harvest}$
3. Test $\text{rhs}(0, 5, 0.5, 10, 1.5, 1)$. What does the answer mean in terms of the system?
4. Test $x = 4$ as well

Dictate how the farm evolves: Solution

1. Write `harvest(x, h, a)` and `growth(x, r, K)`
2. Write `rhs(t, x, r, K, h, a) = growth - harvest`
3. Test `rhs(0, 5, 0.5, 10, 1.5, 1)`. What does the answer mean in terms of the system?
4. Test `x = 4` as well

```
def growth(x, r, K):  
    return r*x*(1 - x/K)  
  
def harvest(x, h, a):  
    return h*x/(a + x)  
  
def rhs(t, x, r, K, h, a):  
    return growth(x, r, K) - harvest(x, h, a)
```

```
print(rhs(0, 5, 0.5, 10, 1.5, 1))    # 0.0  
print(rhs(0, 4, 0.5, 10, 1.5, 1))    # 0.0
```

0.0

0.0

Intelligence Upgrade: Lists vs Arrays

- Lists hold anything and can grow; NumPy arrays hold numbers and do math elementwise
- Sorting: `sorted(lst)` vs `np.sort(arr)`
- `np.linspace(a, b, n)` gives `n` evenly spaced points between `a` and `b`

```
lst = [1, 2, 3]
arr = np.array([1, 2, 3])

np.sort(arr)
arr.sum()

x = np.linspace(0, 12, 121)
```

Intelligence Upgrade: Lists vs Arrays

- Lists hold anything and can grow; NumPy arrays hold numbers and do math elementwise
- Sorting: `sorted(lst)` vs `np.sort(arr)`
- `np.linspace(a, b, n)` gives `n` evenly spaced points between `a` and `b`

```
lst = [1, 2, 3]  
arr = np.array([1, 2, 3])
```

```
np.sort(arr)  
arr.sum()
```

```
x = np.linspace(0, 12, 121)
```

```
<class 'list'>  
<class 'numpy.ndarray'>  
[1 2 3]  
6
```

Multiple farms

1. Make `farm_pop = [4.2, 7.1, 1.3, 9.8, 5.5]` as a list, then convert it to a NumPy array
2. Multiply each by 2, as a list and as an array. What's different?
3. Sort the array from largest to smallest. Print its sum (2 decimals) and max with an f-string
4. Let `x = np.linspace(0, 12, 121)`. Call `rhs` on all of `x` at once (no loop!)

Multiple farms: Solution

1. Make farm_pop = [4.2, 7.1, 1.3, 9.8, 5.5] as a list, then convert it to a NumPy array
2. Multiply each by 2, as a list and as an array. What's different?
3. Sort the array from largest to smallest. Print its sum (2 decimals) and max with an f-string
4. Let x = np.linspace(0, 12, 121). Call rhs on all of x at once (no loop!)

```
pens_list = [4.2, 7.1, 1.3, 9.8, 5.5]
pens = np.array(pens_list)
print(pens_list * 2)
print(pens * 2)
print(np.sort(pens)[::-1])
print(f"Sum {pens.sum():.2f}, max {pens.max()}")
x = np.linspace(0, 12, 121)
rates = rhs(0, x, 0.5, 10, 1.5, 1)
print(rates.shape)      # (121,)
```

```
[4.2, 7.1, 1.3, 9.8, 5.5, 4.2, 7.1, 1.3, 9.8, 5.5]
[ 8.4 14.2  2.6 19.6 11. ]
[9.8 7.1 5.5 4.2 1.3]
Sum 27.90, max 9.8
(121,)
```

Decision Engine: If/else Statements

- Run code only when a condition is True
- Comparisons: < <= == != >= >
- Combine conditions with and / or / not
- Indentation is the syntax (4 spaces)

```
if age < 13:  
    status = "child"  
elif age < 18:  
    status = "teenager"  
else:  
    status = "adult"
```


Decision Engine: If/else Statements

- Run code only when a condition is True
- Comparisons: < <= == != >= >
- Combine conditions with and / or / not
- Indentation is the syntax (4 spaces)

```
if age < 13:  
    status = "child"  
elif age < 18:  
    status = "teenager"  
else:  
    status = "adult"
```

'adult'

Farm triage

1. Write a function `classify(x)` that returns “doomed” if $x < 4$, “viable” if $4 \leq x \leq K$, and “overcrowded” otherwise
2. Test it on $x = 2, 4, 6, 12$
3. **Bonus:** write `trend(x)` that returns “growing” if $\text{rhs}(0, x, r, K, h, a) > 0$, else “shrinking”. Try $x = 4.5$ and 6

Farm triage: Solution

1. Write a function `classify(x)` that returns “doomed” if $x < 4$, “viable” if $4 \leq x \leq K$, and “overcrowded” otherwise
2. Test it on $x = 2, 4, 6, 12$
3. **Bonus:** write `trend(x)` that returns “growing” if $\text{rhs}(0, x, r, K, h, a) > 0$, else “shrinking”. Try $x = 4.5$ and 6

```
def classify(x):  
    if x < 4:  
        return "doomed"  
    elif x <= K:  
        return "viable"  
    else:  
        return "overcrowded"  
  
print([classify(p) for p in [2, 4, 6, 12]])  
  
def trend(x):  
    return "growing" if rhs(0, x, r, K, h, a) > 0 else "shrinking"  
  
trend(x = 4.5)  
  
['doomed', 'viable', 'viable', 'overcrowded']  
'growing'
```

Loop Protocol: For & While Loops

- for repeats a known number of times;
while repeats until a condition fails
- indentation says when loop ends
- Careful: a while condition that never fails
runs forever

```
for k in range(3):  
    print(k)           # 0 1 2  
  
while x > 0.1:  
    x = x - 0.5*x  
  
# one Euler step  
x = x + dt*rhs(0, x, r, K, h, a)
```

Simulate by Predicting the future

1. For loop: start at $x = 6$ with $dt = 0.1$ and take 500 Euler steps of the form
$$x = x + dt \cdot \text{rhs}(0, x, r, K, h, a)$$
2. Print the final population (day 50)
3. While loop: start at $x = 3$ and step until $x < 0.1$, tracking elapsed time. Print the collapse time

Simulate by Predicting the future: Solution

1. For loop: start at $x = 6$ with $dt = 0.1$ and take 500 Euler steps of the form
 $x = x + dt \cdot \text{rhs}(0, x, r, K, h, a)$
2. Print the final population (day 50)
3. While loop: start at $x = 3$ and step until
 $x < 0.1$, tracking elapsed time.
Print the collapse time

```
dt = 0.1
x = 6.0
for n in range(500):
    x = x + dt*rhs(0, x, r, K, h, a)
print(f"Day 50 population: {x:.2f}") # 5.06

x, t = 3.0, 0.0
while x > 0.1:
    x = x + dt*rhs(0, x, r, K, h, a)
    t = t + dt
print(f"Collapse at t = {t:.1f} days") # 16.0
```

```
Day 50 population: 5.06
Collapse at t = 16.0 days
```

Level Up: Basic Linear Algebra

- A matrix is an array of arrays:
`np.array([[a, b], [c, d]])`
- `A @ v` is matrix-vector multiplication (`A * v` is elementwise, not the same!)
- `np.linalg.solve(A, b)` solves $Ax = b$
- `np.linalg.eig(A)` returns eigenvalues and eigenvectors

```
A = np.array([[1, 2],  
              [3, 4]])  
v = np.array([1, 1])  
  
A @ v          # [3 7]  
np.linalg.solve(A, v)  
vals, vecs = np.linalg.eig(A)
```


Level Up: Basic Linear Algebra

- A matrix is an array of arrays:
`np.array([[a, b], [c, d]])`
- `A @ v` is matrix-vector multiplication (`A *`
`v` is elementwise, not the same!)
- `np.linalg.solve(A, b)` solves $Ax = b$
- `np.linalg.eig(A)` returns eigenvalues and
eigenvectors

```
A = np.array([[1, 2],  
              [3, 4]])  
v = np.array([1, 1])  
  
A @ v          # [3 7]  
np.linalg.solve(A, v)  
vals, vecs = np.linalg.eig(A)
```

```
[3  7]
```

```
[-1.  1.]
```

```
eigen_vals [-0.37228132  5.37228132]  
eigen_vecs [[-0.82456484 -0.41597356]  
            [ 0.56576746 -0.90937671]]
```

Inter-Farm Traffic: Migrate between farms

1. Humans shuffle between two farms and the AI restocks: $dp/dt = A p + b$, with $A = \begin{bmatrix} -0.4 & 0.1 \\ 0.2 & -0.3 \end{bmatrix}$ and $b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
2. Compute $A @ \begin{bmatrix} 5 \\ 3 \end{bmatrix}$ (the rates of change)
3. Solve $A p = -b$ for the steady state and check that $A @ p + b \approx 0$
4. Find the eigenvalues of A . Both are negative: what does that mean for the pens?

Inter-Farm Traffic: Solution

1. Humans shuffle between two farms and the AI restocks: $dp/dt = A p + b$, with $A = \begin{bmatrix} -0.4 & 0.1 \\ 0.2 & -0.3 \end{bmatrix}$ and $b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
2. Compute $A @ \begin{bmatrix} 5 \\ 3 \end{bmatrix}$ (the rates of change)
3. Solve $A p = -b$ for the steady state and check that $A @ p + b \approx 0$
4. Find the eigenvalues of A . Both are negative: what does that mean for the pens?

```
A = np.array([[ -0.4, 0.1], [0.2, -0.3]])
b = np.array([1, 1])
print(A @ np.array([5, 3]))
vals, vecs = np.linalg.eig(A)
print(vals)
p = np.linalg.solve(A, -b)
print(p)
print(A @ p + b)
```

```
[-1.7  0.1]
[-0.5 -0.2]
[4.  6.]
[0.00000000e+00  2.22044605e-16]
```

Professional Help: Solving ODEs with SciPy

- `solve_ivp` steps an ODE forward in time for us, a smarter version of our Euler loop
- Inputs: the function $f(t, x, \dots)$, a time span (t_0, t_1) and the initial state $[x_0]$
- `args=(...)` passes the constant parameters into f ; `t_eval` picks the times the solution is evaluated
- Returns `sol.t` (times) and `sol.y` (states): x is `sol.y[0]`

```
from scipy.integrate import solve_ivp

sol = solve_ivp(rhs, (0, 50), [6],
               args=(r, K, h, a),
               t_eval=np.linspace(0, 50, 500))

sol.t
sol.y[0]
```

Let SciPy run the farm

1. Solve our ODE from $x_0 = 6$ over $t \in [0, 50]$. Print the final population
2. Compare it with your Euler answer from the loop task
3. Use a for loop to repeat for $x_0 = 1, 3, 4.5, 6, 9$ and print each final population
4. Which starting populations survive? Where do they settle?

Let SciPy run the farm: Solution

```
t_eval = np.linspace(0, 50, 500)
sol = solve_ivp(rhs, (0, 50), [6], args=(r, K, h, a), t_eval=t_eval)
print(sol.y[0][-1])

for x_0 in [1, 3, 4.5, 6, 9]:
    sol = solve_ivp(rhs, (0, 50), [x_0], args=(r, K, h, a), t_eval=t_eval)
    print(x_0, round(sol.y[0][-1], 2))
```

5.067532057151892

1 0.0

3 0.0

4.5 4.89

6 5.07

9 5.11

1. Solve our ODE from $x_0 = 6$ over $t \in [0, 50]$.
2. Use a for loop to repeat for $x_0 = 1, 3, 4.5, 6, 9$
3. Which starting populations survive? Where do they settle?

Gather Evidence with Matplotlib

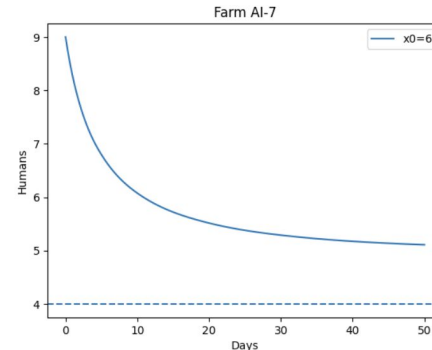
- `plt.plot(x, y)` draws a curve
- optional formatting input: `color='green'`, `marker='o'`, `linestyle='--'`, `label = "curve 1"`
- Always add `plt.xlabel`, `plt.ylabel`, `plt.title`
- If have multiple curves add `plt.legend()` to display labels
- `plt.axhline(y, ls="--")` draws a dashed reference line
- `plt.show()` displays the figure

```
plt.plot(sol.t, sol.y[0], label="x0=6")
plt.axhline(4, ls="--")
plt.xlabel("Days")
plt.ylabel("Humans")
plt.title("Farm AI-7")
plt.legend()
plt.show()
```


Gather Evidence with Matplotlib

- `plt.plot(x, y)` draws a curve
- optional formatting input: `color='green'`, `marker='o'`, `linestyle='--'`, `label = "curve 1"`
- Always add `plt.xlabel`, `plt.ylabel`, `plt.title`
- If have multiple curves add `plt.legend()` to display labels
- `plt.axhline(y, ls="--")` draws a dashed reference line
- `plt.show()` displays the figure

```
plt.plot(sol.t, sol.y[0], label="x0=6")
plt.axhline(4, ls="--")
plt.xlabel("Days")
plt.ylabel("Humans")
plt.title("Farm AI-7")
plt.legend()
plt.show()
```

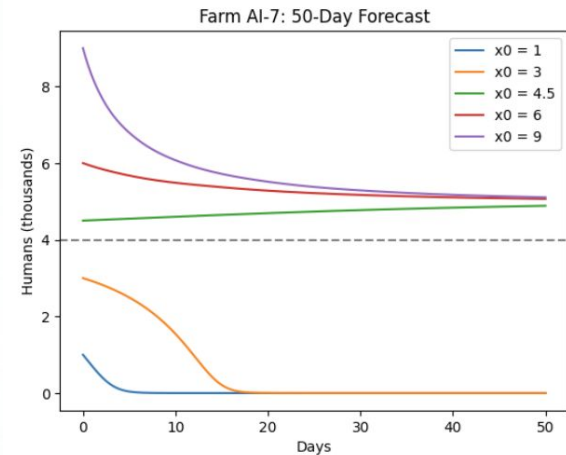


Present Evidence: Show the shareholders

1. Loop over x_0 in $[1, 3, 4.5, 6, 9]$: solve and plot every trajectory on one figure, each with a label
2. Add a dashed line at $x = 4$
3. Add axis labels, a title and a legend, then `plt.show()`
4. Bonus: plot `rhs(0, x, ...)` against `x = np.linspace(0, 12, 121)`. Where does it cross zero?

Present Evidence: Solution

1. Loop over x_0 in $[1, 3, 4.5, 6, 9]$: solve and plot every trajectory on one figure, each with a label
2. Add a dashed line at $x = 4$
3. Add axis labels, a title and a legend, then `plt.show()`
4. Bonus: plot `rhs(0, x, ...)` against $x = \text{np.linspace}(0, 12, 121)$. Where does it cross zero?



```
t_eval = np.linspace(0, 50, 500)
plt.figure()
for x_0 in [1, 3, 4.5, 6, 9]:
    sol = solve_ivp(rhs, (0, 50), [x_0], args=(r, K, h, a), t_eval=t_eval)
    plt.plot(sol.t, sol.y[0], label=f"x0 = {x_0}")
plt.axhline(4, ls="--", color="gray")
plt.xlabel("Days"); plt.ylabel("Humans (thousands)")
plt.title("Farm AI-7: 50-Day Forecast")
plt.legend()
plt.show()
```

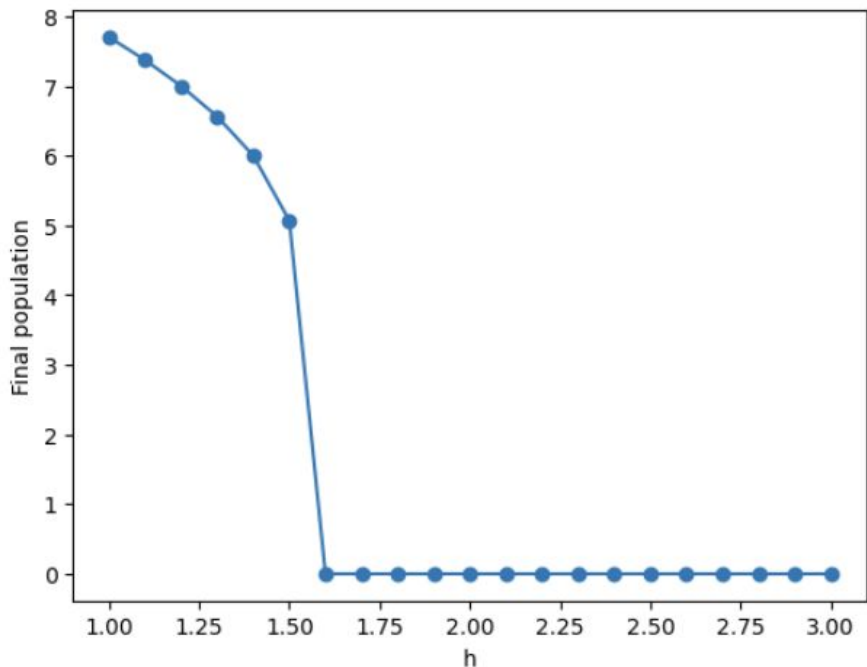
Bonus: Harvest too hard

1. Fix $x_0 = 6$. Sweep h over `np.linspace(1, 3, 21)`
2. For each h , solve the ODE and store the final population in a list
3. Plot final population vs h
4. At what h does the farm collapse?

Bonus: Solution

1. Fix $x_0 = 6$. Sweep h over `np.linspace(1, 3, 21)`
2. For each h , solve the ODE and store the final population in a list
3. Plot final population vs h
4. At what h does the farm collapse?

```
hs = np.linspace(1, 3, 21)
finals = []
for hh in hs:
    sol = solve_ivp(rhs, (0, 50), [6], args=(r, K, hh, a), t_eval=np.linspace(0, 50, 500))
    finals.append(sol.y[0][-1])
plt.plot(hs, finals, "o-"); plt.xlabel("h"); plt.ylabel("Final population"); plt.show()
```



Will humans survive?

● Start above 4

The population survives and settles near 5.

● Start below 4

The population collapses.

⚠ AI harvest warning

Increasing h from **1.5** $\rightarrow$ **1.6** is enough to collapse an otherwise healthy farm.

- You used: imports, data types, f-strings, functions, lists & arrays, if, loops, linear algebra, solve_ivp, matplotlib

A futuristic, light blue and white border surrounds the central text. It features various UI elements: three colored dots (green, green, orange) at the top left; three green dots at the top right; a vertical green line on the left; a vertical orange line on the right; a horizontal orange line at the bottom right; and a horizontal green line at the bottom left. There are also several small circles and lines scattered throughout the border, giving it a high-tech, dashboard-like appearance.

Mission Complete!

**Humanity Saved
Python Unlocked**